@manavcode.ai • AI Engineering Series • 2026

5 AI ENGINEERING PROJECTS

Build These to Get Shortlisted for Any AI Engineering Job

These project types helped me land 2 AI Engineering job offers without company experience.

5 Projects • **Hard** Difficulty • **2** Job Offers • **₹0** Course Cost

Build any 2–3 of these and walk into any AI Engineering interview fully prepared.

HOW TO USE THIS GUIDE

Each project includes:

→ Project overview — what it is in one sentence
→ What it does — the complete feature breakdown
→ Problem it solves — why someone would pay for this
→ Complete tech stack — every tool with its role explained
→ Why interviewers love it — the signal it sends
→ Build tips — how to approach it step by step

Important

All 5 Projects at a Glance

01 **Multi-Agent Research & Report System** — Agent Orchestration + LangGraph
02 **Production RAG + Evaluation Suite** — RAG + Evaluation Engineering
03 **Full Stack AI SaaS Application** — Full Stack + Production Deploy
04 **Autonomous GitHub Code Review Agent** — MCP + Agentic AI + RAG
05 **Personal AI Employee with Memory** — System Design + Memory Systems

PROJECT 1 OF 5

Multi-Agent Research & Report System

A system of 4 specialised agents that researches any topic autonomously and delivers a structured, fact-checked report.

Difficulty: Hard **Type:** Multi-Agent System **Core Skill:** LangGraph + Agent Orchestration

---

WHAT IT DOES

- ✓ Orchestrator agent breaks the topic into 4–6 research subtasks automatically
- ✓ Research Agent searches the web using Tavily and scrapes sources in parallel
- ✓ Analyst Agent extracts and structures key insights from raw scraped content
- ✓ Writer Agent compiles insights into a professional structured report
- ✓ Fact-Checker Agent validates every claim before final output is delivered
- ✓ Human-in-the-loop approval gate — you review before the report is finalised
- ✓ Full LangGraph state management — all agents share state seamlessly

PROBLEM IT SOLVES

Manual research takes 4–6 hours for a comprehensive report. A single LLM call misses depth and cross-referencing. This system replicates what a team of 4 researchers would do in under 10 minutes. Real users: consultants, analysts, journalists, and startup founders.

COMPLETE TECH STACK

- ⚙ LangGraph — state machine for multi-agent workflow orchestration
- ⚙ Claude API (claude-sonnet-4-5) — primary reasoning model for all agents
- ⚙ Tavily Search API — real-time web search (free tier: 1,000 searches/month)
- ⚙ Mem0 — shared memory so agents retain context across the full workflow

WHY INTERVIEWERS LOVE IT

> **Interview Signal**
> *Shows you understand multi-agent architecture — not just single-agent. LangGraph state management, human-in-the-loop design, and parallel agent execution are exactly what Anthropic and OpenAI FDE interviews test. Most candidates build one agent. You built four working together.*

BUILD TIPS

1. Start with 2 agents (Researcher + Writer) before adding Analyst and Fact-Checker
2. Use LangGraph's StateGraph — define a shared AppState dataclass all agents read and write
3. Add the human approval gate last — build the automated flow first
4. Expose it as a FastAPI endpoint so you can demo live in interviews with one API call

---

PROJECT 2 OF 5

Production RAG Pipeline with Full Evaluation Suite

An enterprise-grade RAG system with hybrid search and automated RAGAS evaluation running on every code change.

Difficulty: Hard **Type:** RAG + MLOps **Core Skill:** RAG + Evaluation Engineering

---

WHAT IT DOES

- ✓ Ingests documents from multiple sources — PDFs, URLs, Notion pages, CSV files
- ✓ Hybrid search combining semantic vector + keyword BM25 for better retrieval
- ✓ Re-ranking layer — shortlisted chunks re-ranked by relevance before LLM sees them
- ✓ Query decomposition — complex questions broken into sub-questions automatically
- ✓ RAGAS evaluation suite measuring faithfulness, answer relevancy, context precision and recall
- ✓ GitHub Actions CI/CD — eval suite runs automatically on every pull request
- ✓ Grafana dashboard showing eval scores over time with regression alerts

PROBLEM IT SOLVES

Basic RAG is easy. Production RAG that does not hallucinate, does not degrade over time, and proves it works with measurable scores is what companies need. This project shows you have moved beyond tutorial RAG into properly engineering it.

- Qdrant — vector database with hybrid search support (free self-hosted)
- Claude API — generation model for synthesising RAG responses
- RAGAS — evaluation framework for measuring RAG quality metrics
- LangSmith — traces every RAG call for debugging and monitoring
- GitHub Actions — CI/CD pipeline running automated evals on every push
- Grafana + Prometheus — production metrics dashboard and alerting

WHY INTERVIEWERS LOVE IT

> **Interview Signal**
> *Every candidate builds RAG. Almost nobody can evaluate it. When an interviewer asks how do you know your RAG is working — most candidates say they tested it manually. You show a RAGAS dashboard with faithfulness scores over time. That one answer alone has won offers.*

BUILD TIPS

1. Build basic RAG first — get it returning answers before touching evaluation
2. Set up LangSmith tracing early — makes debugging 10x easier and impresses interviewers
3. Start RAGAS with faithfulness and answer_relevancy only, add more metrics gradually
4. The CI/CD eval pipeline is what elevates this from a project to a production system

---

PROJECT 3 OF 5

Full Stack AI SaaS Application

A complete AI-powered web app with real users, authentication, payments, and production monitoring — not a demo, an actual product.

Difficulty: Hard **Type:** Full Stack Product **Core Skill:** End-to-End Production Deploy

WHAT IT DOES

- ✓ Users sign up with Google OAuth or email — Supabase Auth handles all authentication
- ✓ Free tier with strict usage limits — users see exactly how many credits remain
- ✓ Core AI feature powered by Claude API with real-time streaming responses
- ✓ Credit-based system — users buy more credits via Stripe when they run out
- ✓ Stripe webhook handling — payment confirmed instantly triggers credit top-up

PROBLEM IT SOLVES

Most AI projects are demos. This is a business. It solves a specific problem but the architecture — auth, payments, usage tracking, streaming — applies to any AI SaaS. The AI feature can be swapped. The production infrastructure is permanent and reusable.

COMPLETE TECH STACK

- Next.js 15 (App Router) — frontend with built-in server-side API routes
- Supabase — PostgreSQL database, authentication, Row Level Security
- Claude API — core AI feature with streaming via Server-Sent Events
- Stripe — subscription payments, one-time credits, and webhook processing
- Vercel — zero-config deployment with automatic CI/CD from GitHub
- Tailwind CSS + shadcn/ui — component library for clean fast UI

WHY INTERVIEWERS LOVE IT

> **Interview Signal**
> *You shipped something real. Most candidates have demos with hardcoded fake data. You have a deployed app with actual users, real payments processed, and production monitoring. When asked have you deployed AI to production — you hand them a live URL. That ends the debate.*

BUILD TIPS

1. Build the AI feature working locally before adding authentication or payments
2. Use Supabase Row Level Security — handles multi-tenant data isolation automatically
3. Add streaming responses early — makes the app feel 10x faster and is production-essential
4. Test Stripe in sandbox mode fully before switching to live keys

---

PROJECT 4 OF 5

Autonomous GitHub Code Review Agent

An MCP-powered agent that reviews every pull request automatically and posts structured comments on GitHub.

Difficulty: Hard **Type:** Agentic AI + MCP **Core Skill:** MCP + Tool Use + RAG

WHAT IT DOES

- ✓ Cross-references changes against existing codebase patterns using RAG over your repo
- ✓ Posts structured line-by-line review comments directly on the GitHub PR
- ✓ Learns from feedback — dismissed suggestions are remembered for future PRs
- ✓ Generates a summary comment with overall quality score and top 3 concerns

PROBLEM IT SOLVES

Code reviews take senior engineers 30–60 minutes per PR. At scale this blocks shipping. An AI reviewer handling routine checks frees senior engineers for architecture decisions. This is the exact type of tool FDE engineers build for enterprise clients at Anthropic and OpenAI.

COMPLETE TECH STACK

- ⚙ Claude Code — primary AI agent runtime for code analysis and review
- ⚙ MCP (Model Context Protocol) — custom server connecting Claude to GitHub API
- ⚙ GitHub API + PyGithub — read PRs, post comments, manage webhooks
- ⚙ LangGraph — stateful workflow managing the full review pipeline
- ⚙ ChromaDB — RAG over your existing codebase for pattern matching
- ⚙ Python 3.12+ + FastAPI — webhook listener and API service layer

WHY INTERVIEWERS LOVE IT

> **Interview Signal**
> *MCP is the most in-demand skill in AI Engineering in 2026. Building a custom MCP server that connects Claude to a real external system demonstrates exactly what Forward Deployed Engineers do daily. Interviewers at Anthropic specifically look for hands-on MCP experience. Most candidates cannot show any.*

BUILD TIPS

1. Start with a webhook listener that prints PR diffs — understand the data structure first
2. Build and test the MCP server locally with Claude Code before automating
3. The RAG over codebase layer is optional for v1 — add it after basic review is working
4. Use GitHub sandbox environment to test webhooks without touching real repositories

---

PROJECT 5 OF 5

Personal AI Employee with Memory & Automation

WHAT IT DOES

- ✓ Custom AI persona with specific expertise, name, and communication style via SOUL.md
- ✓ Persistent memory across all sessions — Mem0 remembers every interaction and preference
- ✓ Accessible via Telegram — client messages it exactly like messaging a real team member
- ✓ Scheduled cron jobs — daily briefings at 9 AM, weekly summary reports every Friday
- ✓ 3+ integrated skills — web search, email reading, calendar management
- ✓ Sub-agent spawning for complex tasks — main agent delegates to specialised workers
- ✓ Deployed 24/7 on a VPS with Docker, monitoring dashboard, and auto-restart on failure

PROBLEM IT SOLVES

Small businesses need assistants but cannot afford full-time hires. An AI employee handling research, support, scheduling, and reporting at a fraction of human cost is a real product people pay for monthly. This demonstrates complete system design — architecture, memory, deployment, automation, and monetisation.

COMPLETE TECH STACK

- ⚙ OpenClaw — open-source AI agent runtime wrapping Claude with Telegram interface
- ⚙ Mem0 — persistent memory layer for cross-session context retention
- ⚙ Claude API (Haiku for routine, Sonnet for complex) — cost optimised routing
- ⚙ FastAPI — REST API for skill integrations and webhook handling
- ⚙ Docker + Docker Compose — containerised deployment with resource limits
- ⚙ Telegram Bot API — client-facing interface using python-telegram-bot

WHY INTERVIEWERS LOVE IT

> **Interview Signal**
> *Demonstrates full end-to-end system design thinking — not just building an agent but deploying it as a product someone pays for. Memory architecture, cost optimisation with model routing, security via Docker isolation, and monetisation strategy are all present. This is how senior AI engineers think.*

BUILD TIPS

1. Follow the VPS install method — local installs work for testing but VPS shows production thinking
2. The SOUL.md file is the most important file — invest time on a specific useful persona

YOUR NEXT STEPS

The 3-Step Action Plan

Step 1 — Pick your first project: Don't overthink it. Pick the one that excites you most, not the one that looks most impressive. Excitement drives completion. Start today.

Step 2 — Build in public: Post your progress on Instagram and LinkedIn as you build. 'Day 3 of building my RAG pipeline — here is what I learned.' This builds your brand AND portfolio simultaneously.

Step 3 — Document everything: Write a README, record a 3-minute demo video, note what interviewers would ask. A well-explained project beats a complex unexplained one every time.

> **The honest truth**
> *You don't need all 5. Pick 2–3. Build them properly with good documentation and a live demo. Two strong projects with clear thinking behind them will always beat five half-finished ones. Interviewers tell the difference in 60 seconds.*